C

SUPABASE · REACT · TYPESCRIPT

Codex V1

Technical Documentation

Developer platform & separate Super Admin platform, built entirely on Supabase — PostgreSQL, Auth, Edge Functions, and Row Level Security.

This document describes what was actually implemented, not merely what was planned. Sections mark anything left incomplete explicitly rather than implying full production coverage.

codex-v1 · public beta · free, no payment method required

Contents

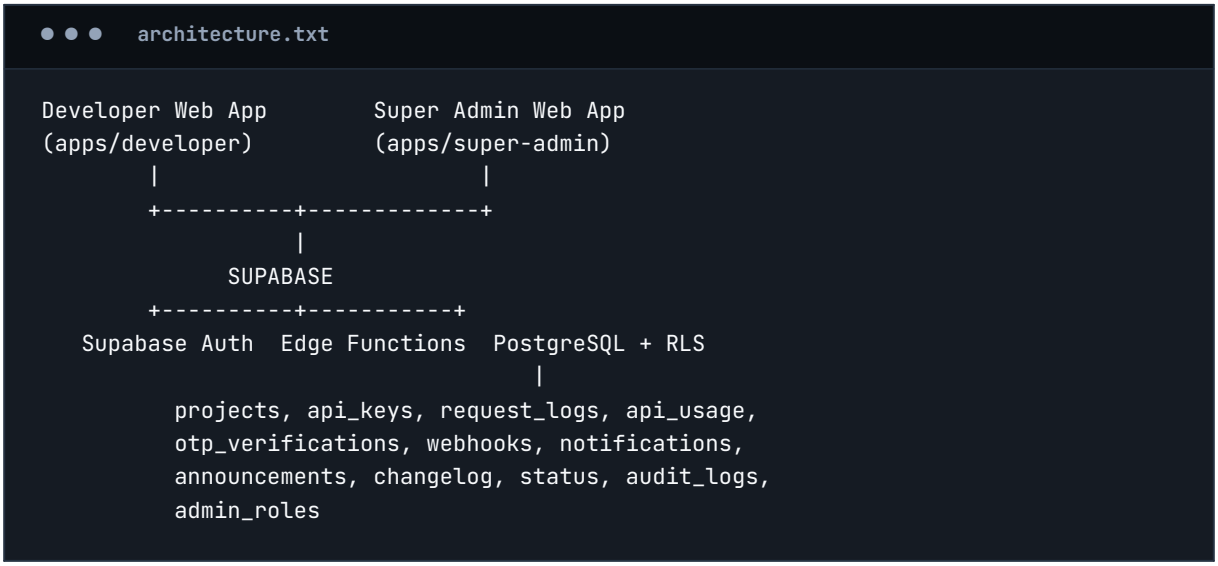
01 Product Overview

Codex is a developer platform providing a versioned utility + OTP API ("Codex API v1"), a developer dashboard for managing projects, API keys, and webhooks, a Developer Tools Hub of ten browser-side utilities, and a physically separate Super Admin web application for platform operators. The public beta is free with no payment method required; billing is explicitly out of scope for V1 but the schema (`organizations.plan`) anticipates it.

02 Architecture

Two independent frontend applications talk to one shared Supabase backend. Neither frontend talks to the other; there is no server-side coupling beyond the shared database and shared Edge Functions.

architecture.txt



Tenant isolation (user → organization → project → API key) is enforced at the database layer via PostgreSQL Row Level Security, not by frontend checks. Changing an ID in a request cannot expose another user's data.

03 Technology Stack

Layer	Choice
Frontend	React 18 + TypeScript + Vite + Tailwind CSS, Recharts, Lucide icons
Backend	Supabase (PostgreSQL, Supabase Auth, Edge Functions, RLS, Storage)
Email	Resend (OTP delivery, announcement email fan-out)

Layer	Choice
Source control / CI	Git + GitHub Actions
Frontend deploy	Cloudflare Pages (two separate projects: codex-developer, codex-super-admin)
Config	.env / .env.example + platform secrets (never committed)

No secondary backend (Node/Express/Fastify/NestJS/Python/Django/Flask/Go/Firebase/separate Postgres) exists anywhere in the codebase. All server-side logic runs as Supabase Edge Functions (Deno runtime).

04 Folder Structure

codex/

```
codex/
  apps/developer/       React app: dashboard, projects, keys, tools, docs
  apps/super-admin/     Separate React app: platform administration
  supabase/migrations/  Numbered, ordered SQL migrations
  supabase/functions/   Edge Functions = the Codex API + privileged ops
  supabase/seed/        Non-sensitive seed data (status components)
  docs/openapi.yaml     OpenAPI spec for implemented /v1 endpoints
  .github/workflows/    CI: lint, typecheck, test, build, deploy
```

05 Database Architecture

PostgreSQL via Supabase. Five ordered migrations, run in filename order:

- **0000_functions.sql** — shared trigger functions (updated_at maintenance, auto-create profile on signup)
- **0001_core_schema.sql** — profiles, organizations, organization_members, projects
- **0002_api_keys_and_usage.sql** — api_keys, request_logs, api_usage, audit_logs
- **0003_platform_tables.sql** — otp_verifications, webhooks, webhook_deliveries, notifications, announcements, changelog_entries, status_components, status_incidents, admin_roles
- **0004_row_level_security.sql** — RLS enabled and policies defined for every table above

06 Database Schema

Table	Purpose
profiles	1:1 with auth.users; is_super_admin convenience flag
organizations / organization_members	Multi-tenant grouping; plan field for future billing
projects	Belongs to an organization; status active/archived/suspended
api_keys	key_prefix (visible) + key_hash (SHA-256, one-way); never stores raw secret
request_logs	Per-request metadata only — never bodies, secrets, or OTP codes
api_usage	Reserved for pre-aggregated daily rollups (not yet populated by a job)
otp_verifications	code_hash (SHA-256); attempt_count / max_attempts; expires_at
webhooks	signing_secret_encrypted (AES-GCM, reversible — see Section 13)
webhook_deliveries	One row per delivery attempt, real or test
notifications	Per-user in-app notifications, fanned out from announcements
announcements	Admin broadcast source; audience + channels
changelog_entries / status_*	Public-facing platform content, admin-writable
audit_logs	who / what / when / context / result for sensitive actions
admin_roles	Explicit Super Admin grant list, separate from profiles

07 Authentication

Developer app: Supabase Auth email/password with signup, email verification, login, logout, password reset, password update, and session persistence via supabase-js. Protected routes redirect to /auth/login when no session exists.

Super Admin app: same Supabase Auth mechanism, but a successful login is not sufficient — see Section 8.

MFA/TOTP: implemented for real using Supabase Auth's client-side auth.mfa.enroll / challenge / verify API (Security settings page). Enrollment produces a scannable QR code and a manual entry key, and requires a valid 6-digit code before the factor is activated.

> **NOTE**
> AAL2 is enforced at sign-in for both apps: ProtectedRoute (developer app) and RequireSuperAdmin (admin app) both check getAuthenticatorAssuranceLevel() and redirect an enrolled-but-unchallenged session to a dedicated MFA challenge page before granting access to anything else.

08 Authorization

Two layers, both real:

- **Database (source of truth):** PostgreSQL RLS policies on every table, driven by two SECURITY DEFINER helper functions — is_super_admin() and is_org_member(org_id) — so policies stay short and consistent across tables.
- **Application (defense in depth):** ProtectedRoute / RequireSuperAdmin components redirect unauthenticated or non-admin users before a page even renders.

The Super Admin app's RequireSuperAdmin specifically queries the admin_roles table using the caller's own session; because RLS on that table only returns rows belonging to the caller, a non-admin user simply sees zero rows rather than being able to probe others' admin status.

09 API Key Architecture

Format: `cx_test_<48 hex chars>` or `cx_live_<48 hex chars>`, generated with crypto.getRandomValues (24 random bytes).

- Created via the api-keys-create Edge Function (never a direct client insert) so secret generation and hashing happen server-side.
- The full secret is returned exactly once in the HTTP response and is never persisted anywhere — only key_prefix (display) and key_hash (SHA-256, for lookup/verification) are stored.

- Rotate: api-keys-rotate generates a new secret for the same key row and overwrites key_hash, invalidating the old secret instantly.
- Revoke: same function, sets status='revoked' and dispatches the api_key.revoked webhook event.
- Scopes: stored as a text array, default ['*']; creation UI accepts a comma-separated scope list.

> **NOTE**
> Scope enforcement was a real gap found and fixed in the follow-up polish pass: hasScope() existed but was called nowhere, making every key's scopes field cosmetic. It's now checked on all 20 /v1/* endpoints (via defineEndpoint()'s scope parameter for the 14 wrapped endpoints, and an explicit check in the 6 direct-pipeline ones) — a key scoped to ['otp'] now genuinely receives 403 PERMISSION_DENIED calling /v1/uuid/generate. Scope names: uuid, hash, base64, json, url, timestamp, validate, regex, otp.

> **NOTE**
> The Super Admin app can see key metadata (prefix, status, last_used_at) but the raw secret is never queryable from anywhere after creation — not even by a database administrator, short of brute-forcing the hash.

10 API Architecture

Every protected /v1/* endpoint is a Supabase Edge Function (Deno) and runs through one shared pipeline instead of reimplementing auth per endpoint:

request-pipeline

```
Incoming Request
  -> authenticateRequest()   key present? valid? active? project active?
  -> checkRateLimit()        120 req / rolling 60s per API key
  -> parse + validate body
  -> execute endpoint logic
  -> logRequest()            safe metadata only, to request_logs
  -> standardized JSON response
```

Simple synchronous utility endpoints (json/url/timestamp/validate/regex) share this pipeline through a single defineEndpoint() wrapper (supabase/functions/_shared/handler.ts) so each endpoint file only contains its own validation + business logic.

11 Complete Endpoint Reference

Endpoint	Notes
POST /v1/uuid/generate	count 1-100, UUID v4
POST /v1/hash	SHA-256/384/512; labeled one-way, not encryption
POST /v1/base64/encode, /decode	UTF-8 safe
POST /v1/json/validate, /format, /minify	Native JSON.parse/stringify
POST /v1/url/encode, /decode, /parse	parse returns protocol/host/path/params
POST /v1/timestamp/convert	unix_seconds / unix_ms / iso8601, bidirectional
POST /v1/validate/email,url,uuid,ip,json,phone	Format/structure only; never claims deliverability
POST /v1/regex/test	Pattern capped 500 chars, input capped 20,000 chars
POST /v1/otp/send, /verify	See Section 12
(internal) api-keys-create, api-keys-rotate	Dashboard-only, Supabase Auth JWT, not an API key
(internal) webhooks (CRUD + test)	Dashboard-only; REST-style routing in one function
(internal) projects-update	Dashboard-only; dispatches project.updated webhook event
(internal) admin-suspend-user	Super Admin-only; Supabase Auth Admin API ban/unban
(internal) send-announcement-email	Super Admin-only; fans out notifications + Resend email

Full request/response schemas: see docs/openapi.yaml in the repository.

12 OTP Architecture

OTP is designed as general-purpose infrastructure, not just for Codex's own login flow.

- 6-digit numeric codes, generated via crypto.getRandomValues, hashed with SHA-256 before storage — the raw code exists only in memory during /v1/otp/send and, in test-mode keys only, in that single response.
- 5-minute expiry, 5 max attempts; once attempts are exhausted the verification is marked 'failed' permanently, even if the correct code is supplied afterward.
- Test keys (cx_test_...) return the code inline for local development. Live keys never do; email delivery goes through Resend.
- Schema anticipates SMS and TOTP channels (otp_verifications.channel) and a future /v1/otp/status endpoint, though only the email channel is implemented in V1.

13 Webhooks

Full CRUD (create/list/get/update/delete) plus an on-demand signed test delivery, implemented as one REST-style Edge Function (supabase/functions/webhooks).

> **NOTE**
> Webhook signing secrets are encrypted with AES-GCM (ENCRYPTION_KEY), not hashed like API keys or OTP codes. API keys and OTP codes only ever need to be verified (does this match?), so a one-way hash is correct. A webhook signing secret is different — Codex must reuse the raw secret to sign every outgoing delivery, so it has to be recoverable server-side. Hashing it (an early mistake caught during this build) would have made real event delivery impossible.

Automatic dispatch is wired into all five defined events via one shared helper (dispatchWebhookEvent): otp.verified, otp.failed, api_key.created, api_key.revoked, and project.updated. Every delivery attempt, automatic or manual test, is logged to webhook_deliveries.

> **LIMITATION**
> Retry/backoff for failed deliveries is not implemented — a failed delivery is logged once and not retried.

14 Rate Limiting

Fixed-window limiter: 120 requests per rolling 60-second window per API key, implemented as a COUNT query against request_logs (supabase/functions/_shared/rateLimit.ts). Exceeding it returns HTTP 429 with error code RATE_LIMITED.

> **LIMITATION**
> Adequate for beta-scale traffic but not a production-grade limiter — no distributed coordination. A Redis/Upstash-backed limiter is the recommended upgrade path before real load.

15 Analytics

All charts are database-backed (Recharts), reading directly from request_logs — no static or fabricated data anywhere.

- Developer dashboard: total/success/failed requests, error rate, active projects/keys (30-day window), area chart of requests over the last 14 days.
- Developer analytics page: top-10 endpoints (horizontal bar), test-vs-live traffic split (donut).
- Super Admin overview: platform-wide developer/project/key counts, 30-day request volume and error rate, 14-day area chart across all organizations.

The api_usage table (pre-aggregated daily rollups) exists in the schema for when live per-request aggregation becomes too slow, but nothing currently writes to it — all current charts compute from request_logs directly, which is fine at beta scale.

16 Request Logging

Every /v1/* call writes one row to request_logs: request_id, project_id, api_key_id, endpoint, method, status_code, response_time_ms, environment, error_code, timestamp. Request bodies, API secrets, OTP codes, and auth tokens are never logged — enforced by construction, since logRequest() only ever receives the specific safe fields listed above, never the raw request body.

17 Developer Dashboard

Real database-backed stats and quick actions (Create Project, Create API Key, API Explorer, Documentation), matching the spec's requested feature set. First-run guidance (create project → create test key → API Explorer → first request → view usage → read docs) is reflected in the quick-actions layout, though no dedicated onboarding checklist/wizard component was built.

18 Developer Tools Hub

All ten specified tools are implemented and functional, entirely client-side (nothing leaves the browser):

#	Tool	Notes
1-3	JSON Formatter / Validator / Minifier	One page, three outputs from native JSON.parse/stringify
4	Base64 Encoder/Decoder	UTF-8 safe via TextEncoder/TextDecoder
5	UUID Generator	1-100 at a time, crypto.randomUUID()
6	Hash Generator	SHA-256/384/512 via Web Crypto
7	Unix Timestamp Converter	Bidirectional; logic extracted to a tested pure function
8	URL Encoder/Decoder	Plus full URL parsing (protocol/host/params)
9	Regex Tester	Live match highlighting
10	Text Diff	LCS-based line diff, extracted to a tested pure function

19 API Explorer

Sends real requests to the live Edge Functions (not simulated): endpoint picker, API key input, a custom-headers editor, editable JSON body, and a response pane showing status code, response time, request_id, and full JSON — plus one-click "Copy as cURL" for the request and "Copy" for the response.

> **NOTE**
> Coverage was a real gap found and fixed in the follow-up polish pass: the Explorer previously listed only 6 of the ~20 implemented /v1/* endpoints (missing every json/*, url/*, timestamp/*, validate/*, and regex/* endpoint entirely). It now lists all 20.

20 Documentation System

Two content types, rendered by the same DocSectionPage: guide-style prose (docsContent.ts) answering specific onboarding questions — what Codex is, test vs. live keys, auth headers, handling responses/errors, rotating/revoking keys, verifying webhook signatures — and a structured per-endpoint API reference (endpointDocs.ts) covering all 20 endpoints, each with Purpose, Authentication, Request fields, a real request/response example, every error code it can return, cURL AND JavaScript examples, a practical-use note, and security notes where relevant. Changelog and Status content are database-backed and editable live from the Super Admin app; the guide/reference docs are TypeScript data files, redeployed with the app rather than edited live.

> **LIMITATION**
> Code samples are cURL + JavaScript only for every endpoint; Python (and other languages) weren't added given time constraints on this pass.

21 Notifications

In-app notifications are fanned out from admin announcements via the send-announcement-email Edge Function: when an admin publishes an announcement with the "In-app" channel checked, one row is inserted into notifications per matching recipient (resolved by audience: everyone/developers = all profiles; free/pro/enterprise = joined through organizations.plan; specific = an explicit user ID list).

> **NOTE**
> This fan-out is the only thing that ever populates the notifications table — without it, the developer app's Notifications page would have stayed permanently empty, which was caught and fixed during this build.

22 Super Admin

A fully separate Vite/React application (apps/super-admin), independently built and deployed (its own Cloudflare Pages project), not a route inside the developer app. Pages: Overview, Developers (search + suspend/reinstate with real status), Projects, API Keys (metadata only), Logs, Audit Log, Announcements (create + broadcast), Changelog (publish entries), Status Page (update component status, log incidents).

> **NOTE**
> Two fixes from the follow-up polish pass: (1) the app's yellow/amber sidebar, login page, and chart identity was replaced with the same blue/cyan accent system as the developer app — amber is now reserved for genuine warning semantics only. (2) The developer-listing page was rebuilt around a dedicated service-role Edge Function (admin-list-developers) rather than a direct client-side RLS query. The original RLS policy (id = auth.uid() OR is_super_admin()) looks correct on inspection and should already return every row to an admin, but a security-sensitive listing page shouldn't depend on that OR-clause being read correctly by every future maintainer — one typo'd AND silently scopes every admin back to seeing only themselves. The new function also joins in real ban status from the Auth Admin API, so Suspend/Reinstate now reflect actual account state instead of always showing both options.

23 Admin Security

The security boundary is authorization, not URL secrecy: RequireSuperAdmin checks a real database row (admin_roles) via the caller's own RLS-scoped session before rendering any admin page — a hidden URL alone would not have protected anything here. The very first Super Admin has to be granted by direct SQL (documented in the README) precisely because admin_roles can only be written by an existing admin, closing the obvious self-escalation path. Suspension uses the real Supabase Auth Admin API (auth.admin.updateUserById with a ban duration), callable only from a service-role Edge Function, never from the browser.

> **NOTE**
> profiles.is_super_admin — a second, unused admin-authority column that always read false and disagreed with the real admin list — was removed in this pass (migration 0005). admin_roles, checked via the is_super_admin() SECURITY DEFINER function, is now the only source of admin authority; it always was in practice, but the dead column was a footgun waiting to mislead whoever eventually read it.

NOTE

MFA (TOTP) enforcement now applies to the Super Admin app too: RequireSuperAdmin checks getAuthenticatorAssuranceLevel() the same way the developer app's ProtectedRoute does, redirecting to /admin-mfa-challenge when a verified factor exists but this session hasn't completed it. There's no MFA enrollment UI inside the admin app itself — factors are enrolled via the developer app's Security settings, since they belong to the underlying Supabase Auth user, not to either app individually.

LIMITATION

Still not implemented: login rate limiting/anomaly detection specifically for the admin app — realistic to add at the Supabase Auth project-configuration level but not built as app code in this pass.

24 Audit Logging

Every sensitive action writes one row to audit_logs with actor_id, actor_type, action, resource_type, resource_id, context (JSON), result, and timestamp. Actions currently logged: api_key.create, api_key.rotate, api_key.revoke, webhook.create, developer.suspend, developer.reinstate, announcement.publish_fanout. Secrets are never included in the context field.

25 Environment Variables

Variable	Scope	Purpose
VITE_SUPABASE_URL	Frontend (public)	Supabase project URL
VITE_SUPABASE_ANON_KEY	Frontend (public)	Supabase anon key
SUPABASE_SERVICE_ROLE_KEY	Edge Functions only	Never reaches the browser; privileged DB access
RESEND_API_KEY	Edge Functions only	OTP + announcement email delivery
ENCRYPTION_KEY	Edge Functions only	AES-GCM key for webhook signing secrets

26 Secret Management

API key secrets: never stored (SHA-256 hash only). OTP codes: never stored (SHA-256 hash only). Webhook signing secrets: stored encrypted, not hashed, because they must be reused (see Section

13). Service role key and Resend key: Edge-Function-only, read via Deno.env.get, never bundled into any frontend build. .env is git-ignored in both apps; .env.example contains placeholders only.

27 Deployment

Frontends build to static output (vite build) and deploy to two separate Cloudflare Pages projects. Both include a _redirects file (/* /index.html 200) for SPA routing. Backend deploys via the Supabase CLI: supabase db push for migrations, supabase functions deploy for all Edge Functions.

28 CI/CD

.github/workflows/ci.yml

```
checkout --> install --> typecheck --> lint --> test --> build
  --> verify build output (index.html present, dist non-empty)
  --> upload-artifact (developer-dist / super-admin-dist)
  --> (main only) download-artifact --> deploy to Cloudflare Pages
  --> (main only) supabase db push + supabase functions deploy
```

Five jobs: developer-app, super-admin-app (each builds, verifies, and uploads its own artifact), deploy-developer, deploy-super-admin (each downloads that exact artifact and deploys it — never rebuilds), and deploy-functions. All deploy jobs are gated on the corresponding build job succeeding and only run on a push to main, never on a pull request.

> **NOTE**
> Two real bugs found and fixed in the follow-up polish pass: (1) the deploy jobs previously did their own fresh checkout on a separate runner and tried to deploy a dist/ folder that only ever existed in the BUILD job's VM — every deploy would have failed with a missing-directory error. Fixed by building once and passing the exact built artifact between jobs via upload-artifact/download-artifact. (2) cache-dependency-path pointed at package-lock.json files that don't exist in this repo (npm install has never been run here) — this would have hard-failed actions/setup-node immediately, before a single dependency installed. Fixed with a package.json-hash-keyed manual cache instead; switching to the built-in npm cache with a real lockfile is a good upgrade once one exists.

29 Testing

Present: smoke tests in both apps, plus real unit tests for two pieces of extracted pure logic — the LCS-based line-diff algorithm (diffLines.test.ts, 5 cases) and timestamp conversion (timestamp.test.ts, 6 cases), both wired into the actual tool pages rather than duplicated logic.

NOT IMPLEMENTED

Automated tests for authentication, RLS policy enforcement, the Edge Function auth/rate-limit pipeline, API key lifecycle, OTP attempt-limiting, and webhook signing do not exist yet. All of this logic is believed correct as written, but none of it has been exercised by a test suite or a live Supabase project.

30 Security

- RLS enforced on every table; no table relies on frontend filtering for access control.
- Two SECURITY DEFINER helper functions (is_super_admin, is_org_member) keep policies short and avoid RLS-recursion bugs.
- Service-role key confined to Edge Functions; never present in any frontend bundle.
- API key + OTP secrets: one-way hashed. Webhook secrets: reversibly encrypted for a documented, deliberate reason.
- Every privileged mutation re-verifies the caller's permission server-side using their own JWT before doing anything, rather than trusting a client-supplied flag.
- CORS is intentionally permissive on the public Codex API, since it's meant for external developer integrations — access control is enforced by the API key check, not by request origin.

31 Landing Page, Branding & Theming

Added in a follow-up polish pass, alongside a round of bug-fixing driven by direct code audit rather than assumption.

Public landing page

/ now serves a real public marketing page (previously it redirected straight to /dashboard regardless of auth state) — hero, capability grid, tools grid, a live-looking security example, docs teaser, and footer. A logged-in visitor hitting / is bounced straight to /dashboard; everyone else sees the marketing page.

Logo

Replaced a plain letter-in-a-box placeholder with an original mark: two angle-bracket chevrons (<>) on a blue-to-cyan gradient chip, self-contained so it reads correctly on both dark and light backgrounds without separate variants. Used as the favicon and in both apps' sidebars and auth pages. Implemented as a small source file duplicated in each app rather than a shared npm workspace package — this repo doesn't have workspace tooling wired up, and a broken cross-package import felt worse than a few duplicated lines; a real packages/shared-ui package remains a reasonable next step.

Real theme switching

Settings > Appearance offers Light / Dark / System, actually persisted (localStorage) and actually functional: the core color tokens were converted from static hex values to CSS custom properties with fully-specified values for both themes, referenced through Tailwind's rgb(var(--x) / <alpha-value>) pattern. Toggling the .dark class re-themes every component using these tokens with no per-component changes needed. Recharts components take hex props directly rather than CSS classes, so chart colors (dashboard area chart, analytics bar/pie charts) are resolved explicitly per theme via a small useIsDarkMode() hook rather than inheriting the CSS variables automatically — a gap caught and fixed in the same pass that introduced theming, not left as a known issue.

> **NOTE**
> The Super Admin app intentionally has no theme toggle — it remains dark-only by design, matching the original scope of the settings work (developer-facing, not admin-facing).

32 Functional Settings

Previously two pages (Account, Security); now six, all backed by real state rather than static UI:

Page	What's real about it
Account	Name change (Supabase Auth metadata); email change using Supabase's built-in dual-confirmation flow (links sent to both old and new address)
Appearance	Real theme switching — see previous section
Security	Password change; genuine TOTP/MFA enrollment via Supabase Auth's auth.mfa API (QR code, manual key, verified before activation); real session sign-out ("other sessions" / "everywhere", both real supabase-js scopes)
Notifications & Privacy	Four real toggles (product emails, security emails, in-app announcements, analytics opt-in) backed by a new user_preferences table
Developer Preferences	Default project, API Explorer default environment — same table
About	Centralized version display (see below)

NOTE

Two gaps flagged after the previous pass are now closed: MFA is enforced at sign-in in both apps (see Section 23), and the announcement-sending pipeline (send-announcement-email) now actually reads user_preferences before fanning out — in_app_announcements gates the notifications-table insert, and email_security_alerts vs. email_product_updates (chosen by the announcement's severity) gates the Resend send. A user with no preferences row yet is treated as opted-in on everything, matching the table's own column defaults, rather than silently excluded.

Centralized versioning

version.ts in each app exports PRODUCT_VERSION = 'V1' unconditionally — Development, Beta, and Production are all V1; moving to production is explicitly not a version bump. Displayed on Settings > About rather than hard-coded in multiple places.

33 Troubleshooting

Symptom	Likely cause
Blank screen on load	Missing/incorrect VITE_SUPABASE_URL or VITE_SUPABASE_ANON_KEY in .env
PROJECT_INACTIVE on every /v1 call	Project status was changed to archived/suspended in Project Settings
Webhook test delivery fails with no signature	ENCRYPTION_KEY not set via supabase secrets set
OTP email never arrives (live mode)	RESEND_API_KEY not set, or sending domain not verified in Resend
Can't reach /admin, redirected to admin-login	Signed-in account has no row in admin_roles
404 on deep-linked route after deploy	Static host missing SPA fallback; confirm _redirects was deployed

34 Known Limitations

In priority order, most significant first:

- **Nothing has been run.** Built without network access across every session: no npm install, no live Supabase project, no executed build/lint/typecheck.
- Automated test coverage is thin outside of two pure-logic unit test files (Section 29).
- Webhook delivery has no retry/backoff — a failed delivery is logged once, not retried.
- Session management is limited to Supabase Auth's real sign-out scopes (other/global); there is no per-device session list, since GoTrue's public API doesn't expose one.
- Announcement audience filters for free/pro/enterprise will match nobody until organizations.plan values are actually set beyond the 'beta' default.
- api_usage (pre-aggregated rollups) exists in the schema but nothing writes to it yet.
- Rate limiting is a naive fixed-window COUNT query, fine for beta traffic, not for scale.
- No dedicated accessibility pass (keyboard nav, screen reader) was done across every page — labels, focus states, and reduced-motion support were addressed as they came up.
- There's no MFA enrollment UI inside the Super Admin app itself — an admin enrolls via the developer app's Security settings (factors belong to the underlying Supabase Auth user, not either app), which is a slightly awkward cross-app dependency worth smoothing out eventually.

35 Future Expansion

- SMS OTP and TOTP channels (schema already anticipates both via otp_verifications.channel).
- MFA enrollment UI inside the Super Admin app itself, so an admin doesn't have to enroll via the developer app.
- Webhook delivery retries with exponential backoff.
- A scheduled job to populate api_usage from request_logs for faster long-range analytics.
- Per-device session listing (would require a custom sessions table populated at sign-in).
- Billing/subscriptions once organizations.plan is actually assigned meaningfully.
- SDKs, mobile apps, an API marketplace, and additional developer tools — all left room for by the current architecture without requiring a rewrite.

36 Version Information

Item	Value
Codex version	V1 (public beta) — centralized in version.ts, displayed at Settings > About
Schema migrations	0000 through 0006
Edge Functions	28 deployable functions (see supabase/functions/)
Frontend apps	2 (apps/developer, apps/super-admin)
Document generated	By Claude, across the assistant sessions that built and then revised the codebase